

ATIVIDADE TESTES AUTOMATIZADOS E COBERTURA DE TESTES

EXPERIMENTO *DEPURAÇÃO DE PROGRAMAS E A PRÁTICA DE INTEGRAÇÃO CONTÍNUA*

Erickson Lima, Marcos Lordello Chaim

Escola de Artes, Ciências e Humanidades | EACH | USP

Teste automatizado é a aplicação de ferramentas de software para automatizar um processo manual conduzido por humanos de revisão e validação de um produto de software.

Projetos modernos em ambientes de DevOps, comumente incluem testes automatizados desde seu início.

TIPOS DE TESTES AUTOMATIZADOS

- **Testes de Unidade:** testar unidades individuais de código, como funções, métodos ou classes, de forma isolada. Este teste foca na menor parte testável de um software, pois foca validar a lógica interna de uma unidade, verificando se ela produz o resultado esperado de acordo com sua entrada.
- **Testes de Integração:** testar a interação e a comunicação entre diferentes unidades ou módulos do sistema. Este teste foca em verificar se diferentes interfaces do sistema se conectam e se comunicam corretamente conforme esperado.
- **Testes de Aceitação:** testar as funcionalidades do software em um nível mais alto, como simular a interação do usuário com a interface. Este teste foca em avaliar se o software atende aos requisitos funcionais especificados. No mercado, muitas vezes o teste de aceitação é chamado de testes funcionais.

- **Testes de Desempenho e Carga:** avaliar o desempenho do sistema em termos de velocidade, escalabilidade e capacidade de lidar com cargas de trabalho elevadas.
- **Testes de Segurança:** identificar vulnerabilidades de segurança no software.
- **Testes de Regressão:** garantir que novas alterações no código não quebrem funcionalidades existentes.
- Dentre outros.

O principal *framework* de teste de unidade para a linguagem *Java* é o *jUnit*. Ele fornece um conjunto de bibliotecas que permitem criar e executar testes de unidade de forma automatizada e padronizada.

É possível instalarmos a dependência do *jUnit* a um projeto *Maven*, desta forma, baixando a biblioteca diretamente do repositório do *Maven Central*.

Ao executarmos algum ciclo de vida do *Maven* que execute testes, por exemplo, *mvn verify* ou *mvn test*, os testes de unidade do projeto desenvolvidos com o *jUnit* são executados.

TESTES DE UNIDADE – CLASSE DE EXEMPLO

Dada a classe abaixo, na sequência vamos ver um teste de unidade dela:

```
1 package com.calculadora;
2
3 public class Distancia {
4     private final double milhasEmQuilometros = 1.609;
5     private final double quilometrosEmMilhas = 0.6214;
6
7     enum TipoConversao {
8         QuilometrosParaMilhas, MilhasParaQuilometros
9     }
10
11     public double Converter(TipoConversao tipo, double valor) throws Exception {
12         double resultado = 0;
13
14         if (tipo == null)
15             throw new IllegalArgumentException("Tipo de conversão não suportado");
16
17         switch (tipo) {
18             case QuilometrosParaMilhas:
19                 resultado = valor * quilometrosEmMilhas;
20                 break;
21             case MilhasParaQuilometros:
22                 resultado = valor * milhasEmQuilometros;
23                 break;
24             default:
25                 resultado = valor * milhasEmQuilometros;
26                 break;
27         }
28
29         return resultado;
30     }
31 }
```

Figura 1: Classe Distancia

TESTES DE UNIDADE – TESTE DA CLASSE DE EXEMPLO

Abaixo está a classe de teste DistanciaTest, referente a classe de exemplo Distancia

```
1  package com.calculadora;
2
3  import org.junit.Test;
4  import org.junit.runner.RunWith;
5  import org.junit.runners.JUnit4;
6  import com.calculadora.Distancia.TipoConversao;
7  import org.junit.Assert;
8
9  @RunWith(JUnit4.class)
10 public class DistanciaTest {
11
12     @Test
13     public void testeConverterMilhasParaQuilometros() throws Exception
14     {
15         Distancia dist = new Distancia();
16         double valor = dist.Converter(TipoConversao.MilhasParaQuilometros, valor:2);
17
18         Assert.assertEquals(expected:3.218, valor, delta:0.001);
19     }
20 }
21
```

Figura 2: Teste da classe Distancia

Observações:

- Acima da definição da classe, na linha 9, está determinado que o teste deverá ser executado com *jUnit4*:

`@RunWith(JUnit4.class)`

- O método que representa um teste, deve ser decorado com o atributo `@Test`, conforme a linha 12.

Os métodos da classe Assert são usados para avaliar os resultados e gerar uma exceção caso o resultado não seja o esperado, existe uma grande quantidade de métodos de validação, seguem alguns exemplos:

- assertEquals: Verifica se dois valores são iguais.
assertEquals(esperado, atual);
- assertTrue: Verifica se uma condição é verdadeira.
assertTrue(condicao);
- assertFalse: Verifica se uma condição é falsa.
assertFalse(condicao);
- assertNull: Verifica se um objeto é nulo.
assertNull(objeto);
- fail: Força um teste a falhar.
fail("Mensagem de falha opcional");

Na engenharia de software, é comum utilizar uma métrica chamada "Cobertura de Testes" para avaliar a extensão em que um conjunto de testes abrange/cobre o código-fonte de um programa. Esta métrica nos fornece uma medida quantitativa para identificarmos código não testado de um programa.

Uma das principais ferramentas de levantamento de cobertura de testes para aplicações desenvolvidas com a linguagem *Java* é a *JaCoCo*.

É possível utilizarmos a *JaCoCo* em um projeto *Java* utilizando o *Maven*, para utilizá-lo é necessário a configuração do plugin no arquivo *pom.xml*:

```
28 <build>
29   <plugins>
30     <plugin>
31       <groupId>org.jacoco</groupId>
32       <artifactId>jacoco-maven-plugin</artifactId>
33       <version>0.8.7</version>
34       <executions>
35         <execution>
36           <id>prepare-agent</id>
37           <goals>
38             <goal>prepare-agent</goal>
39           </goals>
40         </execution>
41         <execution>
42           <id>report</id>
43           <goals>
44             <goal>report</goal>
45           </goals>
46         </execution>
47       </executions>
48     </plugin>
49   </plugins>
50 </build>
```

Figura 3: Configuração do plugin da JaCoCo

Observe que na configuração do plugin, nas linhas 35 e 42, está definido para a JaCoCo ser executada durante a preparação do agente e na geração de relatório:

prepare-agent: O *goal **prepare-agent*** prepara o agente da *JaCoCo* para registrar os dados de execução. Por padrão, os dados de execução são gravados no arquivo *target/jacoco.exec*.

report: o *goal **report*** cria relatórios de cobertura de código a partir dos dados de execução registrados pelo agente da *JaCoCo*. Por padrão, os dados de execução são lidos no arquivo *target/jacoco.exec* e o relatório de cobertura de código é gravado no diretório *target/site/jacoco/index.html*.

COBERTURA DE TESTES - JACOCO - LEVANTAMENTO DE COBERTURA

Com esta configuração, ao executarmos, por exemplo, o comando `mvn test`, será gerado um arquivo `jacoco.exec` no diretório `target`:

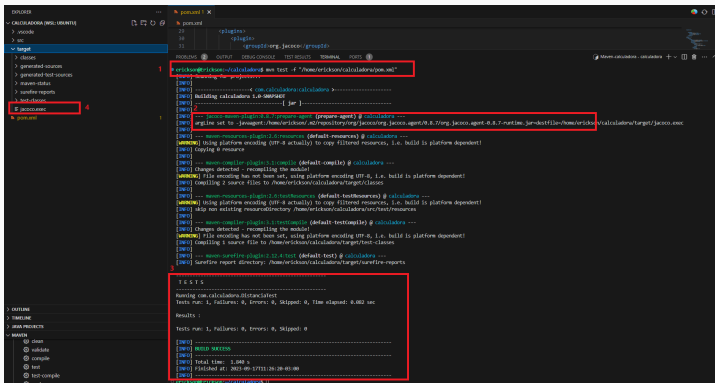


Figura 4: Levantamento de cobertura da JaCoCo

COBERTURA DE TESTES - JACOBO - GERAÇÃO DE RELATÓRIO

Com esta configuração, ao executarmos o comando *mvn jacoco:report*, será gerado o relatório de cobertura da JaCoCo:

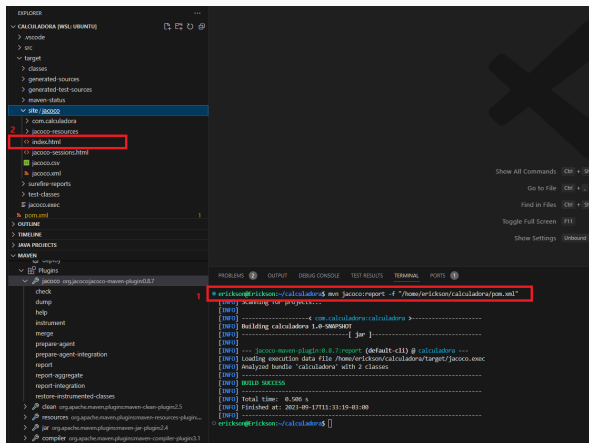


Figura 5: Geração de relatório da JaCoCo

Ao executarmos o comando `mvn clean`, depois o `mvn verify`, também é gerado o relatório.

O relatório foi gerado e pode ser acessado por meio do arquivo `target/site/jacoco/Index.html`:

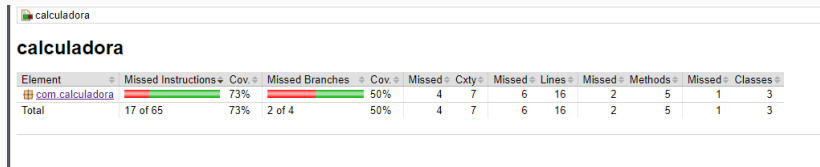


Figura 6: Cobertura do programa calculadora

Observe que o programa está com 73% de cobertura de instruções e 50% de cobertura de ramos (*branches*).

COBERTURA DE TESTES - JACOPO - GERAÇÃO DE RELATÓRIO

calculadora > com.calculadora

com.calculadora

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
Distancia		70%		50%	2	4	3	11	0	2	0	1
Main		0%		n/a	2	2	3	3	2	2	1	1
Distancia.TipoConversao		100%		n/a	0	1	0	2	0	1	0	1
Total	17 of 65	73%	2 of 4	50%	4	7	6	16	2	5	1	3

Figura 7: Cobertura do pacote calculadora

Observe que a classe *Main* está sem cobertura, já a classe *Distancia* possui 70% de cobertura de instruções e 50% de ramos, já o *Distancia.TipoConversao* está totalmente coberto.

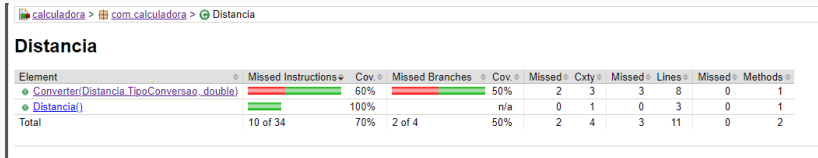


Figura 8: Cobertura da classe Distancia

Observe que o construtor *Distancia()* possui 100% de cobertura, já o método *Converter* possui cobertura de 60% de instruções e 50% de ramos.

COBERTURA DE TESTES - JACOCO - GERAÇÃO DE RELATÓRIO

Ao clicar sobre o método, é exibido o texto da classe, com as linhas cobertas em verde e em vermelho as não cobertas. Também é exibido um símbolo (losango), indicando que aquele ponto é um ramos e fica em amarelo quando não está sendo coberto todas as suas possibilidades.

Distancia.java

```
1. package com.calculadora;
2.
3. public class Distancia {
4.     private final double milhasEmQuilometros = 1.609;
5.     private final double quilometrosEmMilhas = 0.6214;
6.
7.     enum TipoConversao {
8.         QuilometrosParaMilhas, MilhasParaQuilometros
9.     }
10.
11.     public double Converter(TipoConversao tipo, double valor) throws Exception {
12.         double resultado = 0;
13.
14.         if (tipo == null)
15.             throw new IllegalArgumentException("Tipo de conversão não suportado");
16.
17.         switch (tipo) {
18.             case QuilometrosParaMilhas:
19.                 resultado = valor * quilometrosEmMilhas;
20.                 break;
21.             case MilhasParaQuilometros:
22.                 resultado = valor * milhasEmQuilometros;
23.                 break;
24.             default:
25.                 break;
26.         }
27.         return resultado;
28.     }
29. }
30. }
```

Figura 9: Cobertura do código da classe Distancia

Veja a videoaula com mais detalhes:

<https://drive.google.com/file/d/10iCleURjhuZGvTdaGQPVKRL610jjetIG/view?usp=sharing>

Link do repositório utilizado no exemplo: [https:](https://github.com/each2006-2semestre-2023/Calculadora)

[//github.com/each2006-2semestre-2023/Calculadora](https://github.com/each2006-2semestre-2023/Calculadora)

LISTA DE TAREFAS DA ATIVIDADE

1. Assistir videoaula sobre teste automatizado utilizando jUnit e cobertura de testes utilizando a ferramenta JaCoCo. Estudar outras referências se for necessário.
2. Repetir os passos descritos na videoaula para o exemplo calculadora disponível no repositório:
<https://github.com/each2006-2semestre-2023/Calculadora>
3. Alterar a esteira CI/CD que vocês criaram para incluir a análise de cobertura de instruções e ramos (*branches*) para o exemplo *Biblioteca* utilizando a ferramenta JaCoCo.
4. Inclua testes automatizados adicionais de forma a aumentar a cobertura de teste.
5. **Desafio:** é possível obter cobertura de 100%? Inclua um arquivo texto (txt) explicando a sua resposta.